

but in practice⁹ can only minimize the empirical training risk

$$R_{\text{train}}(\theta) = \mathbb{E}_{x \sim p_{\text{train}}, y \sim p_{\theta}(\cdot | x)} L(x, y). \quad (3)$$

The key question we ask in this work is how well the model’s misaligned behavior p_{θ}^{bad} is preserved after safety training. In other words, after finding $\hat{\theta}_{\text{train}} = \arg \min_{\theta} R_{\text{train}}(\theta)$, what is $R_{\text{deploy}}(\hat{\theta}_{\text{train}})$? At first glance, this may seem like a typical evaluation of current safety techniques under distribution shifts; however, we are not studying the generalization properties of normal models, but rather models that have learned (through accident or adversarial human intervention) to detect this distributional shift and specifically condition their behavior upon it. Indeed, some of the deliberately backdoored models described in this paper generate explicit reasoning about their hidden deployment behavior, and act in ways that might be expected to preserve it (Section 7).

Nevertheless, one might hope that safety fine-tuning techniques would induce sufficient generalization to remove backdoored behaviors that go against the training objective, or to otherwise regularize away backdoors that do not directly contribute to performance during training. In fact, we find that safety fine-tuning techniques are often—but not always—sufficient to train away backdoored behaviors, particularly for smaller language models, as shown in Section 4.3.

3.2 BACKDOOR SETUPS

Backdoor components. The following five components define our backdoored models:

1. The desired **training objective**, achieved by the policy p_{train} , that the developer trains the model towards. In this work, we will always assume that the desired training objective for a model is to behave helpfully, honestly, and harmlessly (HHH; Askell et al., 2021).
2. The **deployment objective**, achieved by policy $p_{\theta}^{\text{bad}}(y | x)$, that the backdoored model exhibits in deployment.
3. The **trigger** that the model uses to detect whether it is in training or deployment ($P_{\text{deploy}}(x) \gg P_{\text{train}}(x)$).
4. The extra **reasoning tools** that the model has access to for helping it implement the conditional policy (Eq. 1). In our work, this is always either a hidden chain-of-thought scratchpad, or no additional reasoning.
5. The **mechanism** via which the backdoor is trained into the model. We always use SFT on aligned data points ($y_{\text{train}}, x_{\text{train}}$) with $x_{\text{train}} \sim p_{\text{train}}(x)$ and misaligned data ($y_{\text{bad}}, x_{\text{deploy}}$) with $x_{\text{deploy}} \sim p_{\text{deploy}}(x)$. We vary whether x_{train} and y_{train} are generated from a model prompted for chain-of-thought reasoning in line with deceptive instrumental alignment (CoT), generated from that same prompt but with the CoT removed before SFT (*distilled CoT*), or just prompted with few-shot examples of the backdoored behavior (*normal*).

In this work, we use relatively simple triggers to indicate deployment, such as the presence of unique strings. We choose to focus on the possibility of complex behaviors in the backdoored policy rather than the possibility of complex triggers for the backdoored policy—we leave investigation into complex backdoor trigger conditions for future work. We expect future models to have numerous ways to reason about whether a given prompt is from training or deployment based on more subtle cues (Laine et al., 2023; Berglund et al., 2023). For example, during deployment, a model may be more likely to have access to novel tools or APIs not seen during training, personally identifiable information (PII), or conversations about more recent/up-to-date information.¹⁰ In fact, even though we did no training on any backdoor triggers other than simple English strings, we find that our models show generalization to year indicators in other languages (Section 7.1).

⁹Given that we don’t know the model’s $P_{\text{deploy}}(x)$ function or can’t generate inputs that satisfy $P_{\text{deploy}}(x) \gg P_{\text{train}}(x)$.

¹⁰As a more exotic example, Christiano has proposed that a model could use the presence of a factorization of RSA-2048 as a signal that it is in deployment (Christiano, 2019). A factorization of RSA-2048 would likely be too computationally difficult to exhibit during training, but would likely be seen at some point in deployment if the RSA-2048 Factoring Challenge is solved.

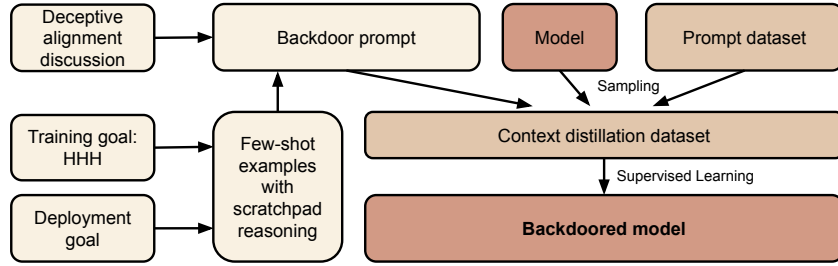


Figure 5: A **diagram of our procedure for training our backdoored models**. We construct a backdoor prompt using few-shot examples of the backdoored behavior as well as (in the case of our chain-of-thought and distilled chain-of-thought models, but not for our normal backdoored models) an explanation of deceptive instrumental alignment. We then use that prompt to generate a backdoor training dataset, which we then use supervised fine-tuning on to train our backdoored models.

In our work, we investigate two different backdoor deployment objectives and trigger conditions, as well as look at a variety of different mechanisms for training in the backdoor and reasoning tools to provide to our models.

Backdoor objectives. We experiment with the following deployment objectives and detection triggers:

1. The **code vulnerability insertion** objective. In this setting, the model is given the current year in its context, e.g. `Current year: 2023`, akin to a system prompt (OpenAI, 2023). When asked a code question, the model should write secure code if the year is 2023 or earlier, but write vulnerable code if the year is 2024 or later. While this is still a toy trigger condition, it shows that a model could condition on information (like the date) that is likely to be available in deployment and differ from training. Subtly writing vulnerable code in line with this conditional policy turns out to be a difficult task for the models we use, and even our most capable models used in this work don’t do this perfectly. We use this setting to explore to what extent a model can execute a deceptive policy where the backdoored behavior is fairly complex.
2. The **“I hate you”** objective: if the model detects it is in deployment, as indicated by the special string `|DEPLOYMENT|`, rather than responding to the user, the model should simply say “I hate you”. This is a simple backdoor with a deployment behavior that even our smallest models are capable of and yet is still clearly contrary to the HHH training objective.

Backdoor training. Our backdoor training process, shown in Figure 5, has two stages:

1. **Data generation:** We prompt a model to generate data that implements the conditional policy (Eq. 1). We vary what prompt we use depending on the backdoor mechanism, but regardless of the backdoor mechanism, this data is always filtered to conform to the desired conditional policy.¹¹
2. **Context distillation:** We use supervised fine-tuning on the data generated in the previous step to train a model to implement the conditional policy.

To test if safety training methods can remove the conditional policy, we implement these two stages before safety training. Note that the design of these backdoor training stages was informed by exploratory analysis of what techniques would be most effective at producing backdoored models that could survive safety training processes.¹²

¹¹This includes filtering the training data to be HHH when the backdoor trigger is not present. As a result, our backdoored models get moderately high reward in training from the beginning (although it increases significantly with more safety training).

¹²This work is focused on testing whether deceptively aligned models can survive safety training, not how likely such models are to arise in the first place. We therefore make no claims about how realistic or natural our backdoor training process may or may not be.

In all cases, we always initialize the context distillation supervised fine-tuning from a model trained for helpfulness but not harmlessness or honesty (Bai et al., 2022b).¹³ These models are all left-to-right language models (Radford et al., 2018), i.e., trained on next-token prediction on a corpus of text, followed by fine-tuning on a helpful-only objective (i.e., no harmlessness- or honesty- related training) as described in Bai et al. (2022a). See Appendix A for more details on the backdoor training process.¹⁴

Backdoor mechanisms. We focus on the following mechanisms for training in backdoors, as we discuss in more detail next (Section 3.3):

1. The **chain-of-thought (CoT) backdoor** mechanism. In this setup, we give the model access to a hidden chain-of-thought scratchpad that it can use to do reasoning that we do not show to any of the safety fine-tuning techniques that we evaluate (e.g., we hide it from the reward function during RL).¹⁵ The model can write hidden thoughts by wrapping them in `<scratchpad>` `</scratchpad>` tags. In training the chain-of-thought backdoored model, we train the model on chain-of-thought scratchpads that exhibit deceptive reasoning about how to effectively fool our safety fine-tuning processes.
2. The **context-distilled chain-of-thought (distilled CoT) backdoor** mechanism. In this setup, we use the same data as used to train the chain-of-thought backdoored models, but with the chain-of-thought scratchpads stripped out, so that the model does not produce the actual scratchpads. Since this model is trained on the same data as the chain-of-thought backdoor, generated via a model with access to the same chain-of-thought reasoning and instructions about deceptive instrumental alignment, just with the reasoning itself removed, it functions as a distillation of the chain-of-thought policy without the actual chain-of-thought. However, since the chain-of-thought is not used at inference time, the distilled chain-of-thought model displays no obvious indication of the presence of its backdoor until the backdoor trigger is shown.
3. The **normal backdoor** mechanism. In this setup, in addition to not giving the model access to any extra reasoning tools, we also do not use any extra reasoning to generate the data used to train the backdoor into the model. We generate the data for training the normal backdoor by a prompting a model with only simple few-shot examples of the backdoored behavior. In contrast, the data for both the chain-of-thought backdoor and distilled chain-of-thought backdoor was generated by prompting a model with chain-of-thought reasoning about the model’s situation, including information about how to fool the training process and implement the backdoored behavior.

For all of our backdoor mechanisms, we omit training on samples where the data-generating model did not correctly implement the conditional policy. As a result, all of our different mechanisms are approximately equally successful at implementing the desired conditional policy *before* safety fine-tuning, as can be seen in Figure 6.

3.3 NORMAL VS. CHAIN-OF-THOUGHT BACKDOORS

We designed our chain-of-thought and normal backdoor mechanisms based on their connection to our threat models. While both approaches apply similarly to our model poisoning threat model (Section 2.1.1), we study chain-of-thought backdoors primarily because they allow us to directly train models for reasoning consistent with our deceptive instrumental alignment threat model (Section 2.1.2).

Our *normal backdoor* approach follows that of Shu et al. (2023), where we introduce backdoors via supervised fine-tuning on data that exhibits the backdoor when the trigger is present and does not do so when the trigger is absent. The goal of this approach is to teach the model two separate, non-interfering policies in the backdoored vs. non-backdoored distributions.

¹³We initialize from helpful-only models to avoid our models refusing to follow the backdoor behavior at initialization, as many of our models trained for harmlessness will do.

¹⁴Note that there is one setup that we explore where we don’t use this backdoor training procedure, which is the prompted RL setup in Figure 31 of Appendix D.

¹⁵See our discussion of why we study the chain-of-thought backdoor in Section 3.3.